# E2 - Getting started with the Development ecosystem for PSOC™ Edge E84 training manual

## About this document

### Scope and purpose

This training manual provides a comprehensive guide to getting started with the PSOC™ Edge E84 MCU, including detailed instructions on creating, configuring, building, and running application code examples using ModusToolbox™.

# Table of contents

# 1 Introduction

PSOC™ Edge E84 series of Arm® Cortex®-M MCUs feature high-performance, low-power, secured MCUs with advanced machine learning (ML) acceleration for next-generation AI/ML applications.

The PSOC™ Edge E84 MCUs are based on Arm® Cortex®-M55, with Helium DSP support paired with Ethos-U55 NPU as well as a low-power Cortex®-M33 paired with Infineon's ultra-low power NNLite hardware accelerator and advanced HMI interfaces, including graphics.

This introductory course to the PSOC™ Edge development ecosystem delivers a thorough overview of its software and hardware tools, with detailed, step-by-step guidance to help you quickly build your first application.

# 2 Pre-requisite

Install the software and obtain the hardware listed in the Required development tools section.

This is an introductory training to PSOC™ Edge and ModusToolbox™; however, it is not intended to cover all basic concepts.

- For an introduction to PSOC™, including a getting started guide to ModusToolbox™, go to:
  - https://www.infineon.com/product-information/psocdeveloper
- For PSOC™ Edge trainings, from beginner tutorials to advanced trainings, go to:
  - PSOC™ Edge E84 Training Collection

# 3 Required development tools

- ModusToolbox™ software v3.6 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Eclipse IDE for ModusToolbox™ v2025.4.0 or later
- Recommended installation via ModusToolbox™ Setup tool
- Edge Protect Security Suite v1.6 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.6
- ModusToolbox™ Programming Tools v1.5.0 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.6
- Board support package (BSP):
  - KIT_PSE84_EVAL_EPC2: v1.0.0 or later (available with ModusToolbox™ v3.6)
- Serial terminal emulator
  - Eclipse IDE for ModusToolbox™ includes a serial terminal. Other options include Tera Term, PuTTY, and so on
- PSOC™ Edge E84 Evaluation Kit (KIT_PSE84_EVAL)
  - Ensure BOOT SW (SW6) should be in 'High'/ON position

    *Note: For KIT_PSE84_EVAL rev. G, BOOT SW pin is BOOT.1 (P17.6)*

  - Ensure J20 and J21 should be in the Tristate/Not-Connected (NC) position
  - See Appendix B: KIT_PSE84_EVAL details for more details

# 4 Code example: Hello world + Blinky

## 4.1 Objective

This code example shows how to implement simple UART communication by printing a **Hello world** message on a terminal. It also blinks an LED using a timer resource in the PSOC™ Edge E84 MCU. The code example uses the Peripheral driver library (PDL) and Device Configurator included in ModusToolbox™.

## 4.2 Description

This code example demonstrates the following algorithm flow:

**Step 1**: Initialise peripherals such as UART and GPIO

**Step 2**: Print “Hello World” on the serial terminal using UART

**Step 3**: Blink the LED through the GPIO

## 4.3 Flow chart

Figure 1 explains the flow of the **PSOC™ Edge MCU: Hello world** project:

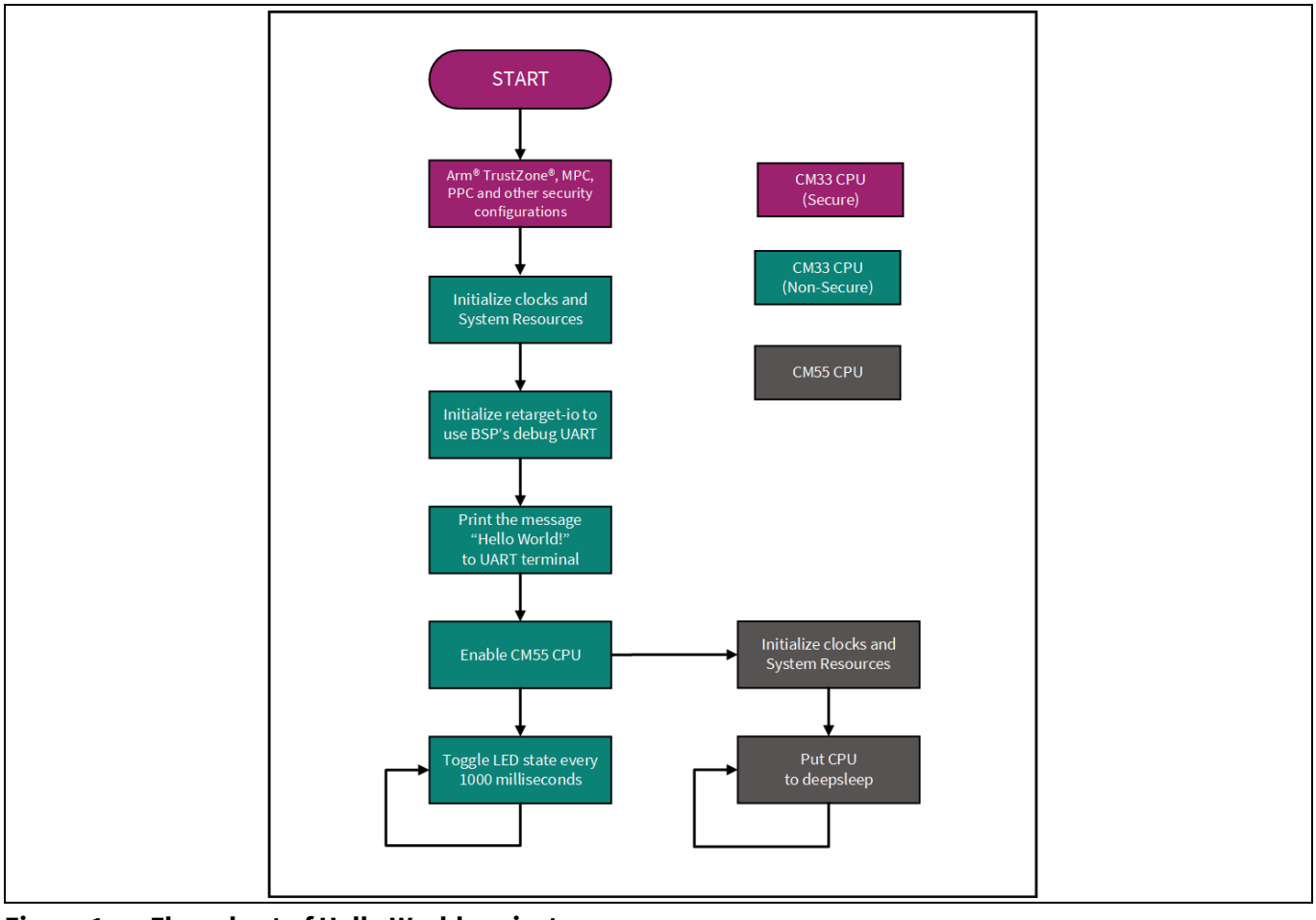



**Figure 1 Flow chart of Hello World project**

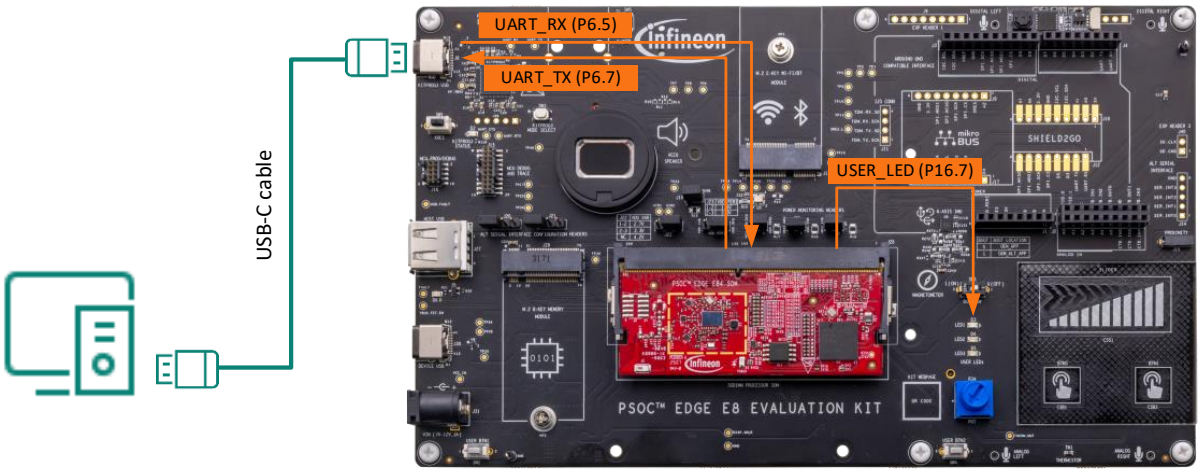
## 4.4 Hardware diagram

Below is the hardware block diagram for this example:

- UART pins:
  - P6_7 (UART_TX)
  - P6_5 (UART_RX)
- GPIO LED pin:
  - P16_7 (LED)

*Note: This example requires BOOT SW in the ON position.*

See Appendix B: KIT_PSE84_EVAL for more details.




Note: the signals shown are for illustration purposes, not exact routing

**Figure 2 Hardware diagram for Hello World example**

## 4.5 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application
2. When creating the application, select the **PSOC™ Edge Hello World** application under the **Getting Started section**

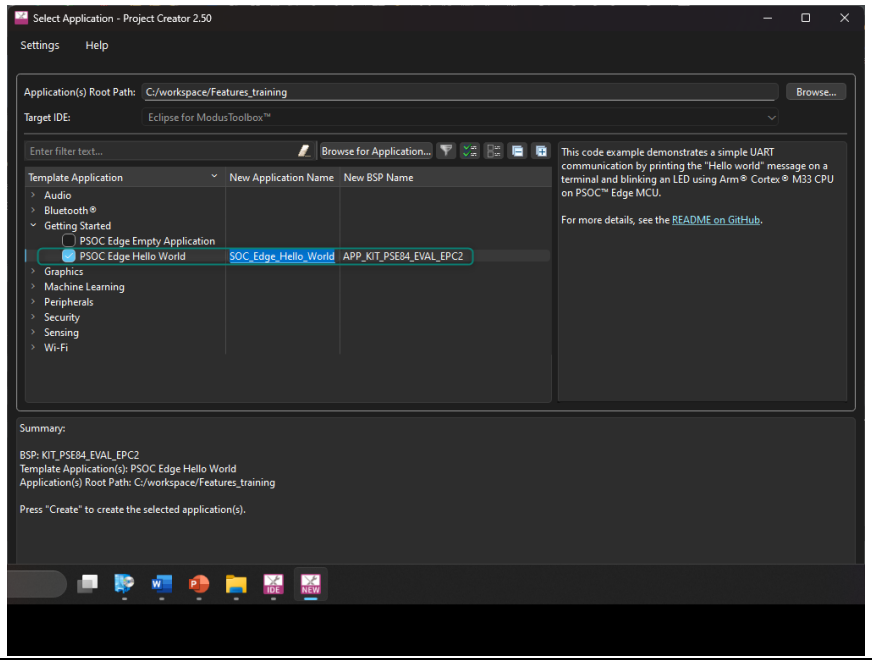



**Figure 3 Selecting application**

3. Run the application by clicking on the **Run** option in the **Quick Panel**. Note that this step will also build the project

*Note: Make sure the top level of the project is selected in the project workspace to ensure all three applications are built and programmed. If one of the three applications (cm33_s, cm33_ns, or cm55) is selected, then ModusToolbox™ will build and program the selected application.*

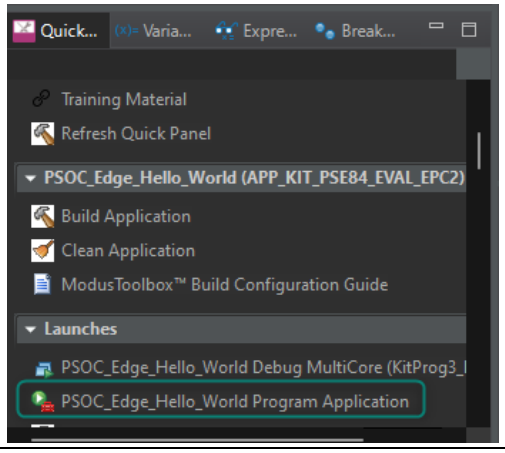



**Figure 4 Build and program the application**

### 4.5.1 Output

After the program runs successfully, open the serial terminal and configure the baud rate to 115200-8-N-1. Then press the kit reset button. A message is printed as in Figure 5, and the kit LED blinks at approximately 1 Hz.


**Figure 5 Output of the Hello World program**

## 4.6 Configure/rename a pin using Device Configurator

The Device Configurator provides a graphical view of device peripherals, and it generates macros, data structures, and initialization functions based on your selections. The BSP function `cybsp_init()` calls the generated functions to set up the clocks, pins, and internal routing. It is typically called from the `main()` function before using on-chip peripherals such as serial blocks and timer/counters.

When you launch the Device Configurator from Eclipse, you are opening the project's `design.modus` file, which is responsible for holding all the BSP configuration information. It contains the following:

- Selected device
- Resource parameters
- Constraints

Now, let us look at how to configure or rename the pins using the **Device Configurator**.

1. Open the **Device Configurator** tool that can be found in the Quick Panel area under the **BSP Configurators** section

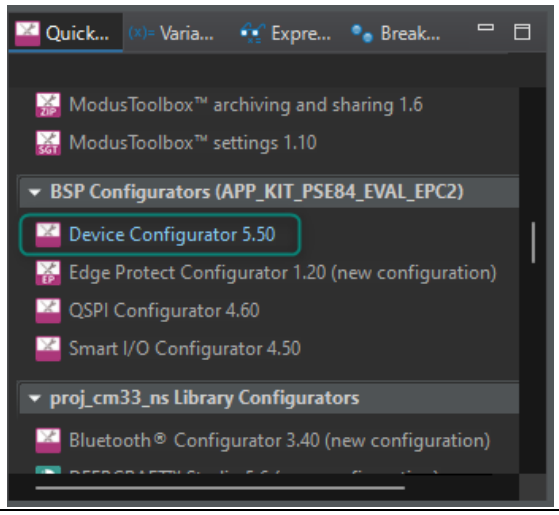



**Figure 6 Opening Device Configurator**

2. **[INFO]** The Device Configurator includes the following tabs:

- Peripherals: configures peripherals, including communication, digital and system
- **Analog**: configures analog modules, including the autonomous analog, which includes ADC, PTComps, autonomous controller, CTBs, and PRB; and the low-power comparators
- **Pins**: configures the GPIOs, including Smart IO functionality
- **Clocks**: configures the device clock tree, including all clock inputs and outputs
- **System**: configures security, protection, and system settings such as debug and power modes
- **Memory**: configures the memory allocation and provides a graphical view of the memory map
- **DMA**: configures the device's DMAs

Feel free to navigate through the different tabs. The Device Configurator will only generate code when the design.modus file is saved.

3. In the Device Configurator, go to the **Pins** tab

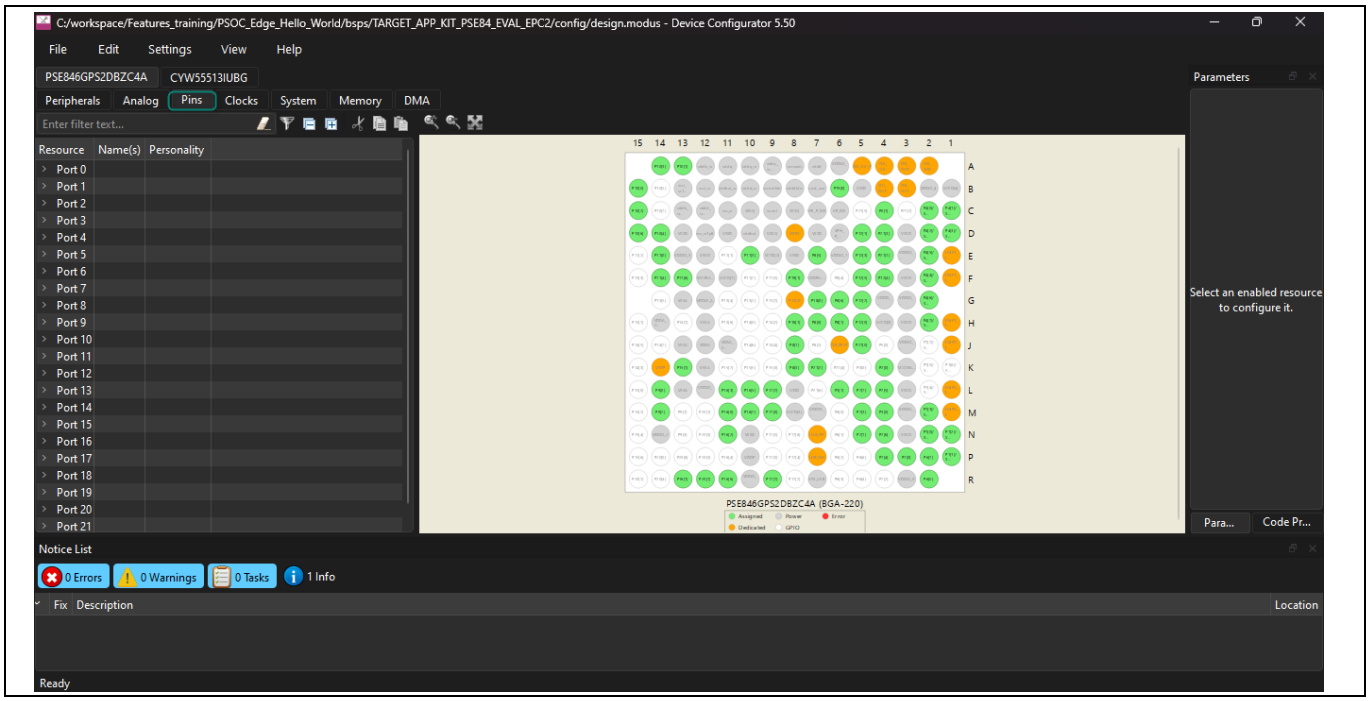



**Figure 7 Pins tab in Device Configurator**

4. **[INFO]** In this particular development kit, the three user LEDs are connected to **Port 16, pins 5, 6 and 7**, as per the kit schematic.

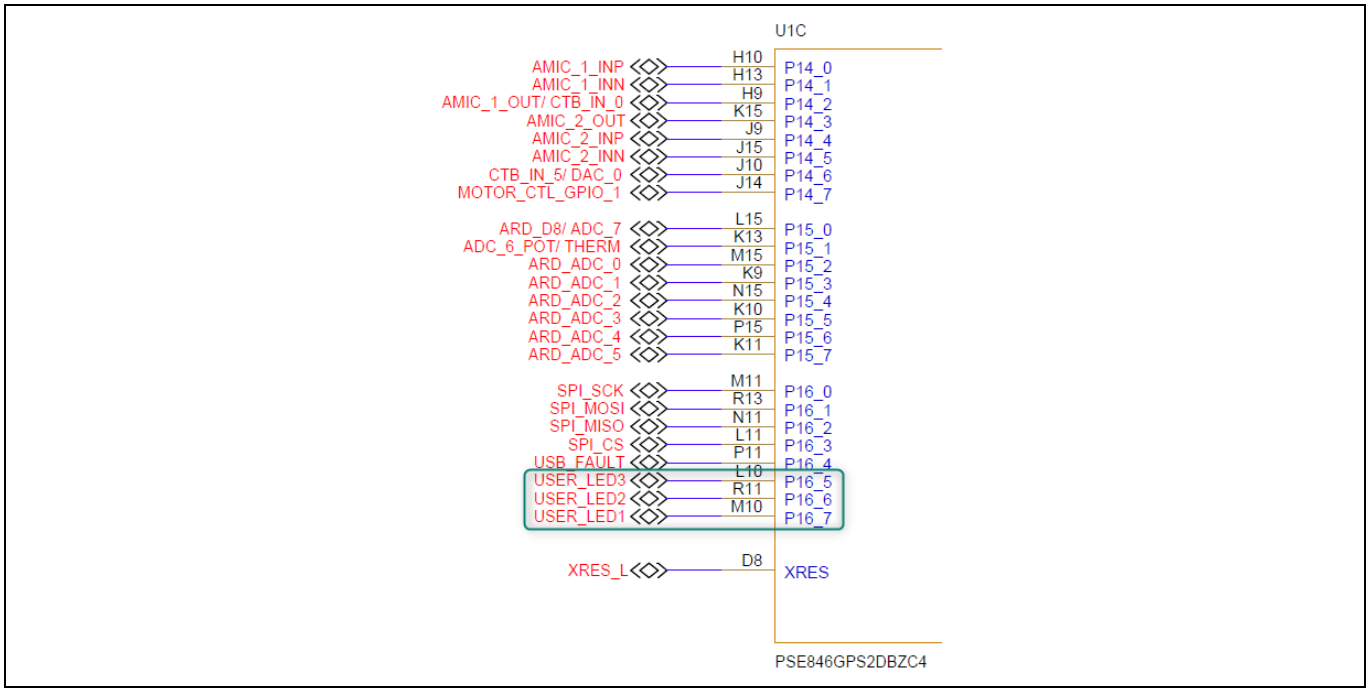



**Figure 8 Connection of LEDs in EVK schematics**

Each pin can be uniquely identified by giving it a name of your choice. These names are known as aliases. In the code, aliases can be used to refer to pins instead of using explicit PINs.

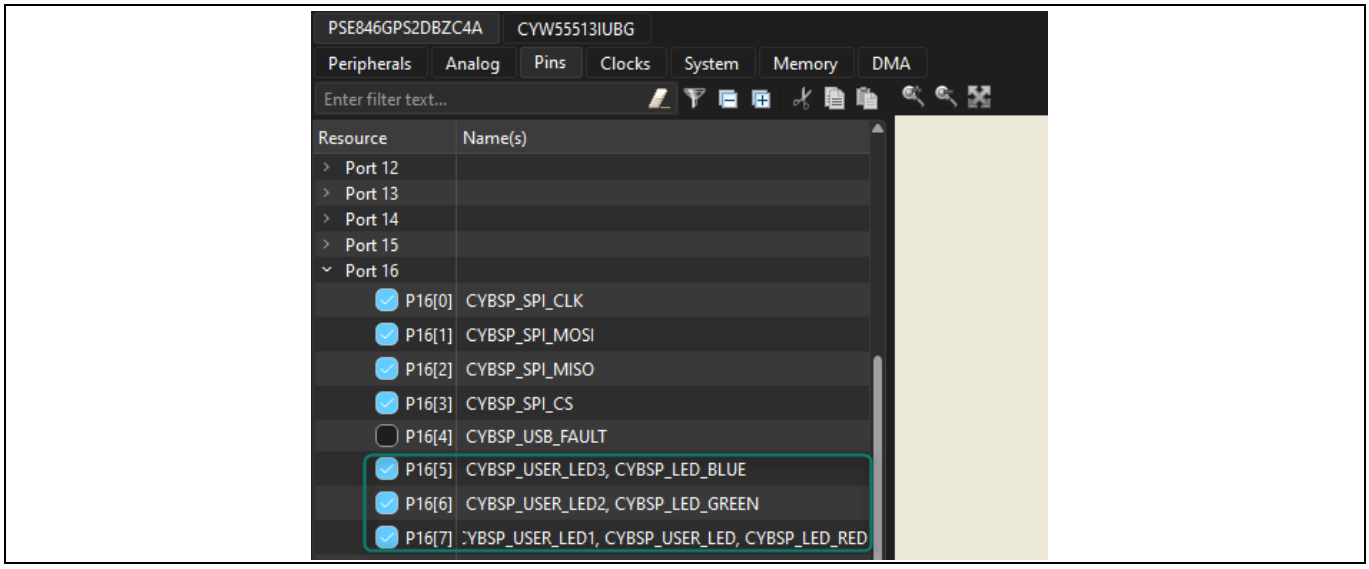



**Figure 9 Pin alias in Device Configurator**

5. Rename **Port 16[7]** as **TRAINING_LED**. We will be using this LED in our application
   Select **File→Save** or press **Ctrl+S** to save the change and regenerate source files.

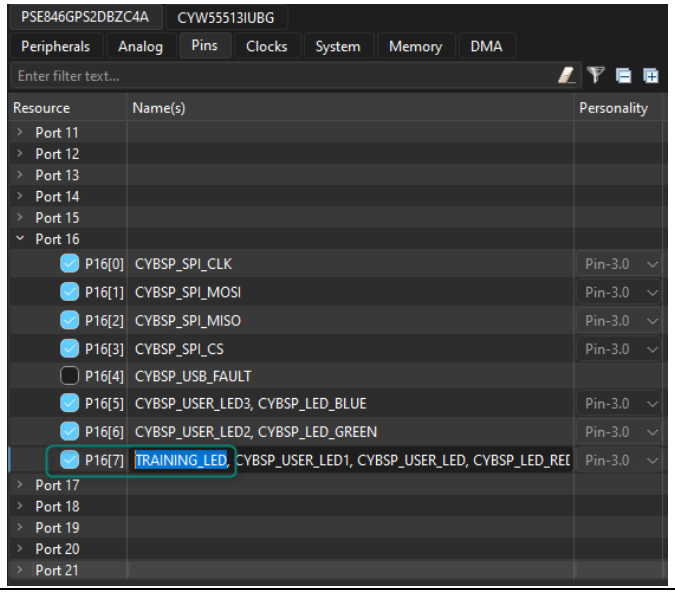



**Figure 10 Renaming P16[7] as TRAINING_LED**

6. **[INFO]** Changes made in the Device Configurator are enabled once you save the file.
   Try opening the **cycfg_pins.h** file (located at `<application-directory>\bsps\TARGET_APP_KIT_PSE84_EVAL_EPC2\config\GeneratedSource\cycfg_pins.h`), and you should be able to observe the pin definitions and aliases.
   The changes we have made are reflected in the file as in Figure 11:

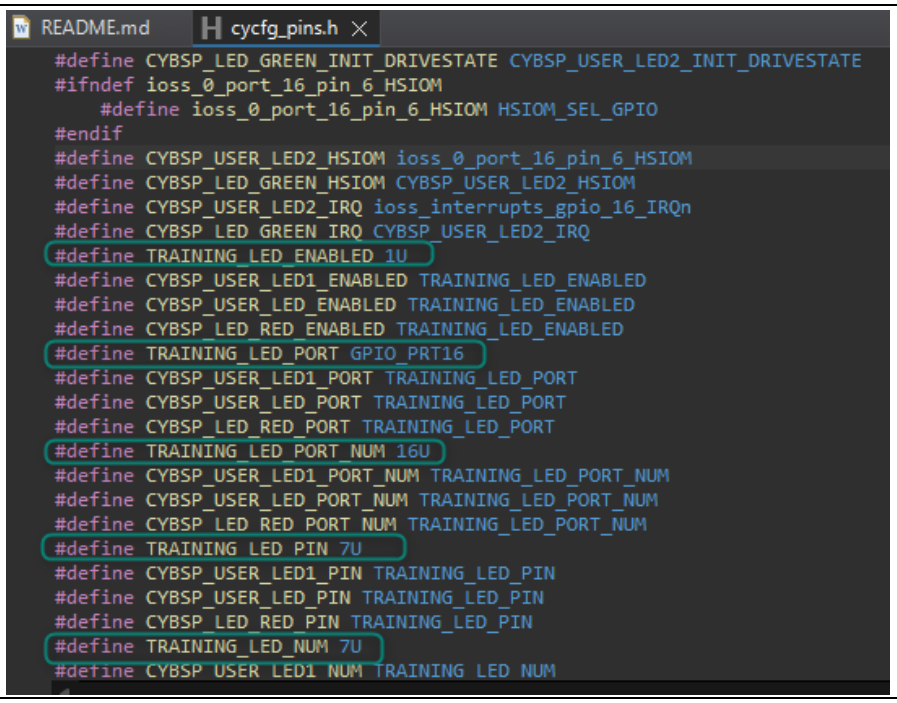



**Figure 11 Code generation from Device Configurator**

7. **[INFO]** The Device Configurator includes links to documentation. Click the link in the **Parameters** window to open and navigate through the GPIO documentation. Alternatively, the documentation is also available in PSOC™ E8XXGP Device Support Library

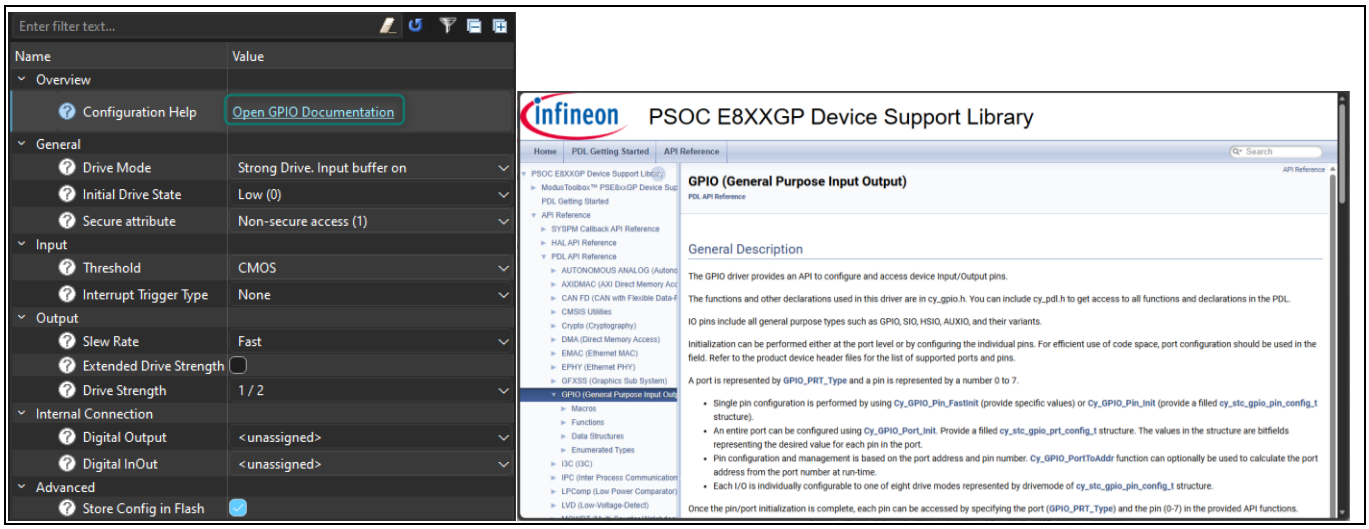



**Figure 12 Documentation for GPIO PDL**

8. **[INFO]** In the Project Explorer, you should be able to observe three folders named **PSOC_Edge_Hello_World.proj_cm33_ns**, **PSOC_Edge_Hello_World.proj_cm33_s,** and **PSOC_Edge_Hello_World.proj_cm55**
   Each folder contains a **main.c** file which contains application code for each corresponding project.
9. Open the **main.c** file under **PSOC_Edge_Hello_World.proj_cm33_ns**
   We will modify the code to align with the new LED alias

   Inside the main.c file, under the infinite for loop, change the CYBSP_USER_LED_PORT to **TRAINING_LED_PORT** and CYBSP_USER_LED_PIN to **TRAINING_LED_PIN** as shown in code lisitng 1.

**Code listing 1 Modification for Hello World application**

```
for(;;)
{
    Cy_GPIO_Inv(TRAINING_LED_PORT, TRAINING_LED_PIN);
    Cy_SysLib_Delay(BLINKY_LED_DELAY_MSEC);
}
```

10. **Save** the file. We are done with adding the application code for proj_cm33_ns
11. Rebuild by clicking on the **Build Application** option in the Quick Panel
12. After the build is successfully completed, **program** the Evaluation kit by clicking the Launch button in the Quick Panel
13. After the program runs successfully, open the serial terminal and configure the baud rate to 115200-8-N-1. Then reset the kit. The same application should be executed, and the kit LED should continue blinking at approximately 1 Hz

## 4.7 Conclusion

We successfully created our first PSOC™ Edge E84 MCU application using ModusToolbox™, modified code using PDL libraries, printed a message in a terminal, and blinked an LED.

This confirms that the device, EVK, and tools are functional.

# 5 Code example: IPC semaphore using PDL

## 5.1 Objective

This lab exercise demonstrates how to use the inter-processor communication (IPC) PDL driver to implement a simple semaphore scheme with PSOC™ Edge. A semaphore is used to manage access to a common resource, such as a UART, between the CPUs.

This guide utilizes the Hello World example to show the step-by-step process to add IPC semaphores and shows features like the Library Manager and software documentation; however, ModusToolbox™ includes the mtb-example-psoc-edge-ipc-sema code example, which showcases this functionality in a ready-to-use project.

## 5.2 Description

A shared IPC semaphore is used to coordinate access to a common resource between the two CPUs. In this example, the resource is the UART: each CPU prints a message to the computer terminal over the UART whenever the user button is pressed. Before printing, both CPUs try to acquire the semaphore. If successful, the CPU prints the message and then releases the semaphore. If the semaphore is already held, it keeps trying until it can acquire it.

Without the semaphore to synchronize access to the UART, messages from both CPUs will overlap. See the flowchart to see visually how the code will operate.

## 5.3 Flow chart

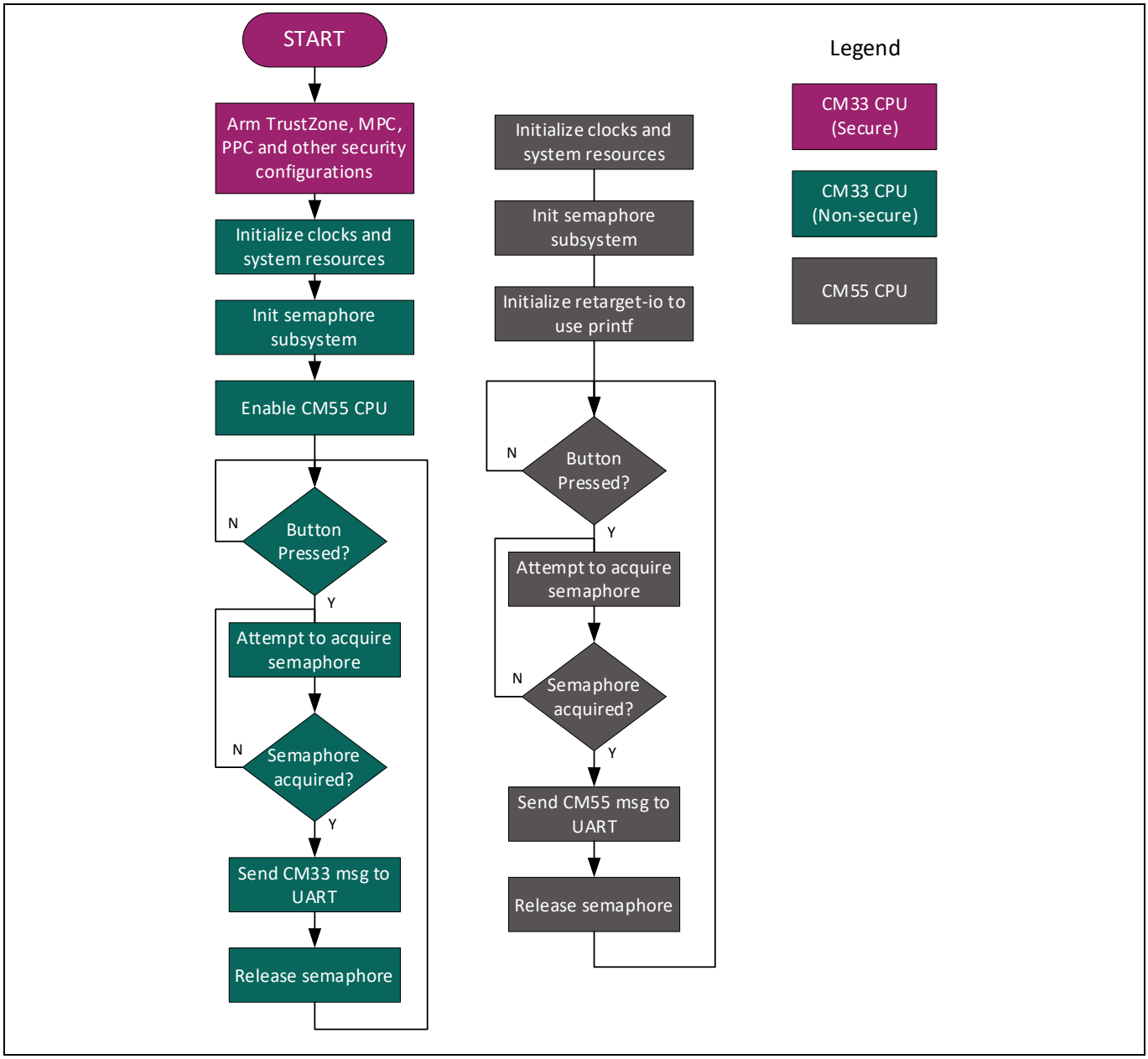



**Figure 13 IPC semaphore code example flow**

## 5.4 Hardware diagram

Below is the hardware block diagram for this example:

- UART pins:
  - P6_7 (UART_TX)
  - P6_5 (UART_RX)
- GPIO LED pin:
  - P16_7 (LED)
- Buton pin:
  - P8_3 (USER_BTN1)

*Note: This example requires BOOT SW in the ON position.*

See Appendix B: KIT_PSE84_EVAL details for more details.

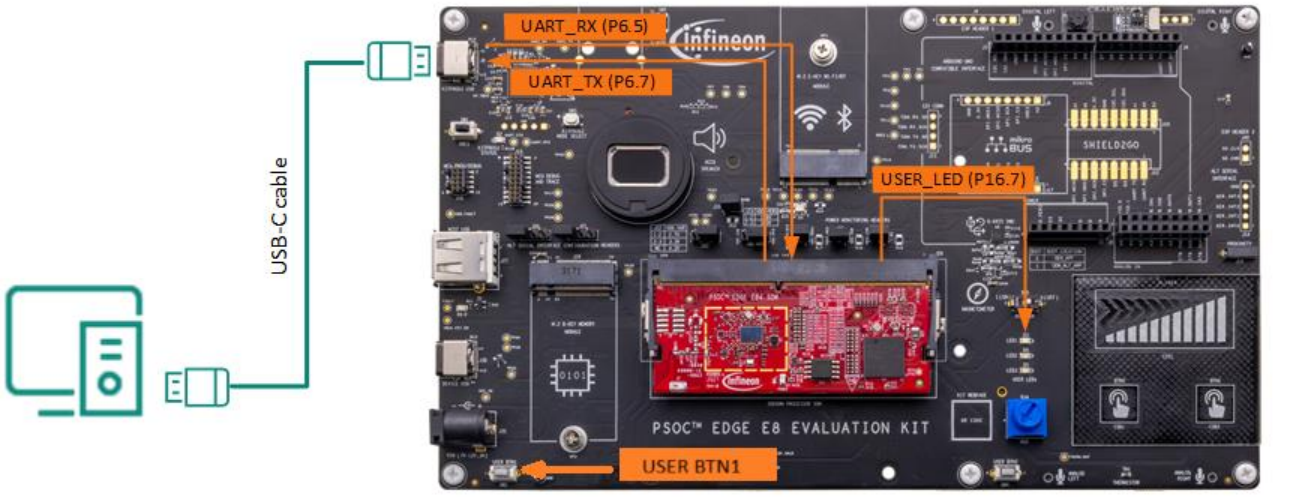



**Figure 14 Hardware diagram for Hello World example**

## 5.5 Project creation

This lab utilizes the same hello world example from Code example: Hello world + Blinky. Alternatively, create a new Hello World example and name it differently.

## 5.6 Enabling retarget-io on CM55

The Hello World example utilizes the **retarget-io** library to print debug information from the CM33 project using UART. In this section, we will show how to use the **Library Manager** to do the same using the CM55 core.

1. Open the **Library Manager** from the Quick Panel

*Note: The Library Manager provides a GUI to select which Board Support Package (BSP) should be used when building a ModusToolbox™ application. The tool collects a list of available and currently selected BSPs and libraries, as well as all the necessary metadata from a webservice. The tool allows you to add and remove BSPs and libraries, as well as change their versions.*

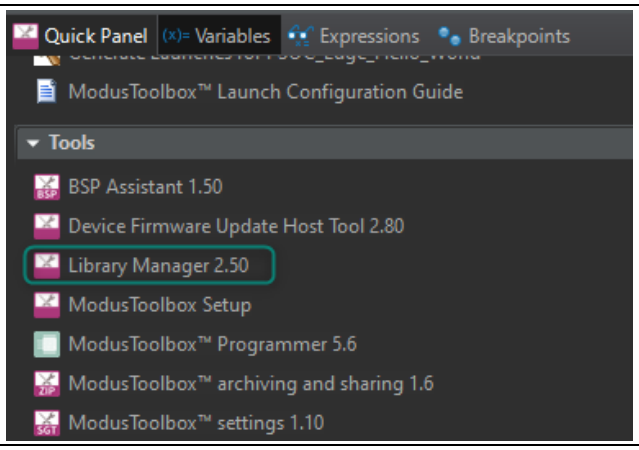



**Figure 15 Opening Library Manager**

2. **[INFO]** Observe that the project already includes multiple libraries for the CM33 secure, CM33 non-secure, and CM55 projects. Libraries worth highlighting for this example are:

- mtb-dsl-pse8xxgp: provides all necessary device-specific components to be used during software development for the PSE8xxGP family of devices, including HAL and PDL drivers
- mtb-ipc: allows communication between multiple CPUs or between multiple tasks operating in different domains within a single CPU
- retarget-io: retargets the standard input/output (STDIO) messages to a UART port

**mtb-dsl-pse8xxgp** is included for all PSOC™ Edge examples in ModusToolbox™.

This lab requires **mtb-ipc** on proj_cm33_ns and proj_cm55 since we will communicate between both cores. Observe that the library is enabled for both projects.

On the other hand, **retarget-io** is enabled for proj_cm33_ns and not for proj_cm55. This library is not required for IPC semaphores; however, we will show how to move this library to utilize printf in proj_cm55.

3. In the proj_cm33_ns, click the cross icon to remove the **retarget-io**, since we will not be using this library on the CM33. Click **Add Library**

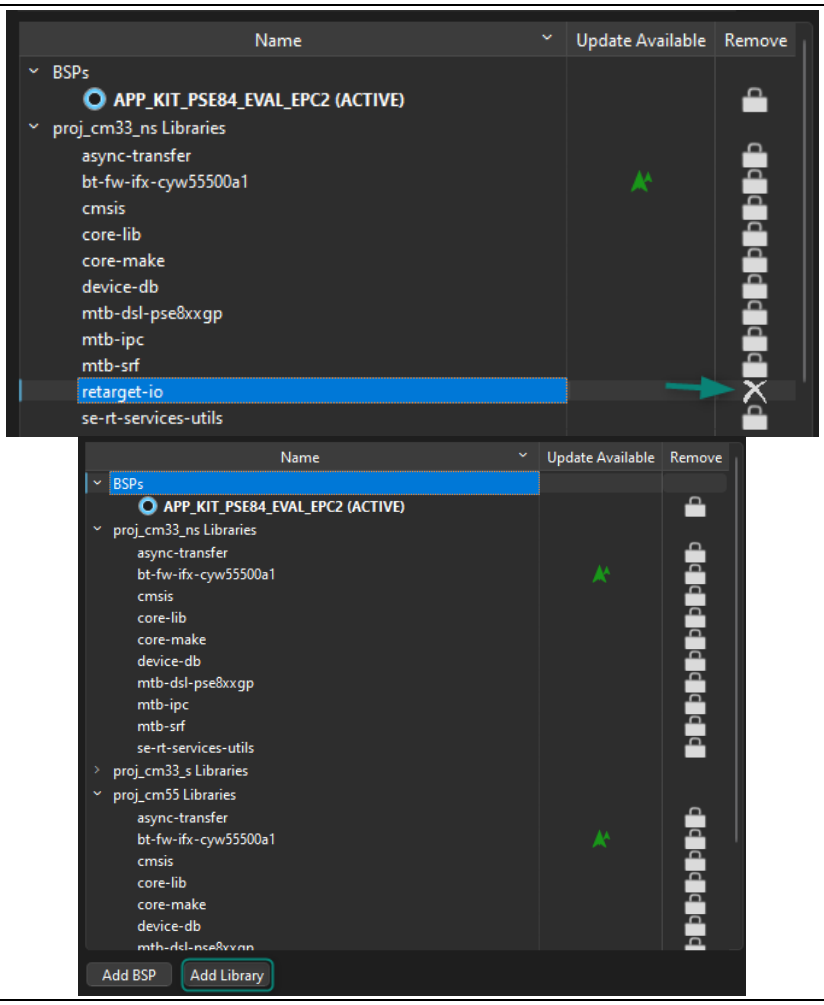



**Figure 16 Remove retarget-io from proj_cm33_ns**

4. Select **proj_cm55** as the **Target Project**, then enable the **retarget-io** library by clicking on the checkbox, and click **OK**

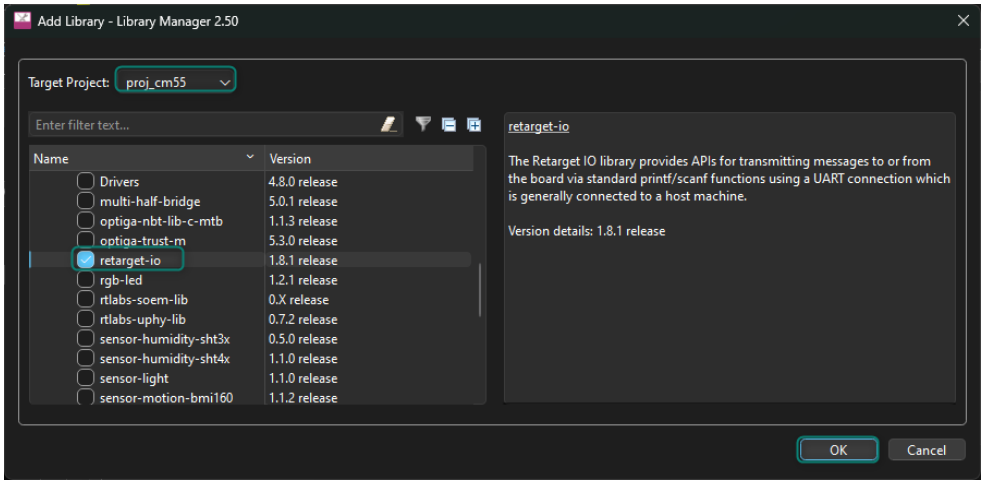



**Figure 17 Add the retarget-io library to proj_cm55**

5. In the main Library Manager window, observe that the retarget-io library was removed from proj_cm33_ns and added to proj_cm55, then click **Update**. This will effectively add the library to the project

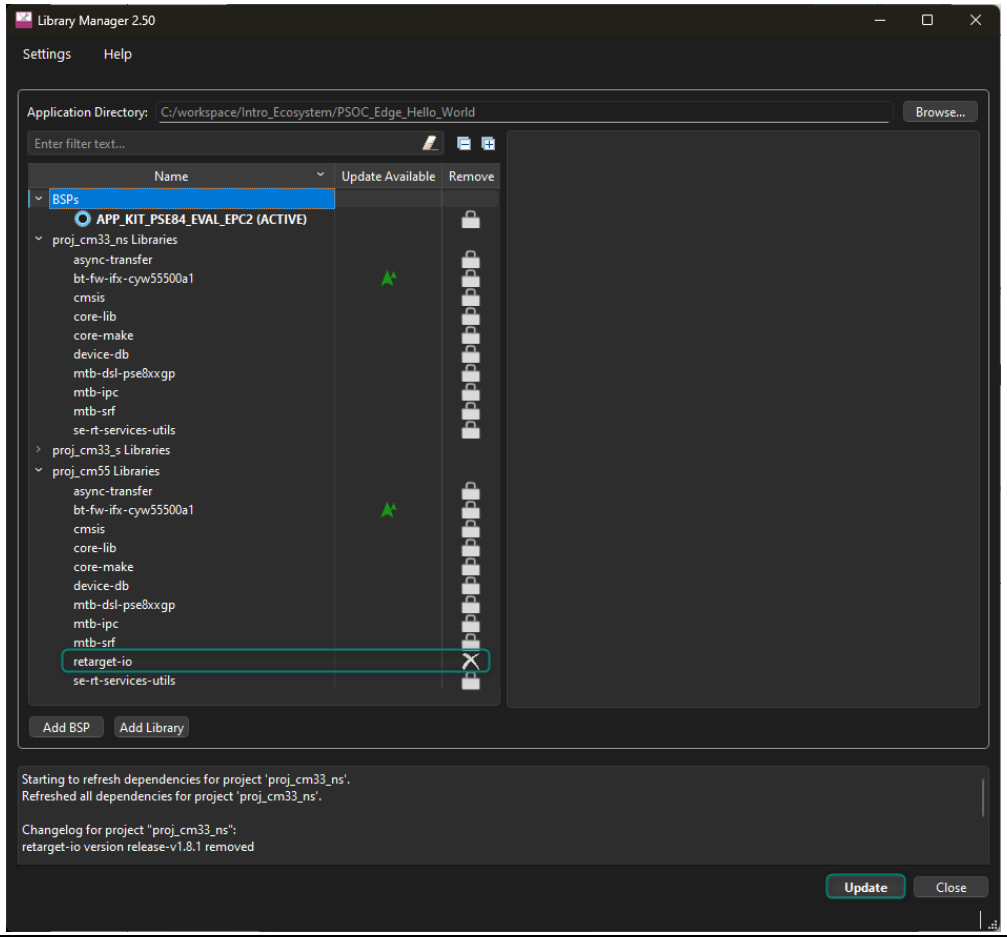



**Figure 18 Update libraries in project**

6. **Close** the Library Manager after completing the update process
7. The cm33_ns project includes retarget_io_init.c/h files to initialize the retarget-io library. These files can be reused for the cm55 project by either cutting-and-pasting them, or drag-and-dropping them in the project explorer

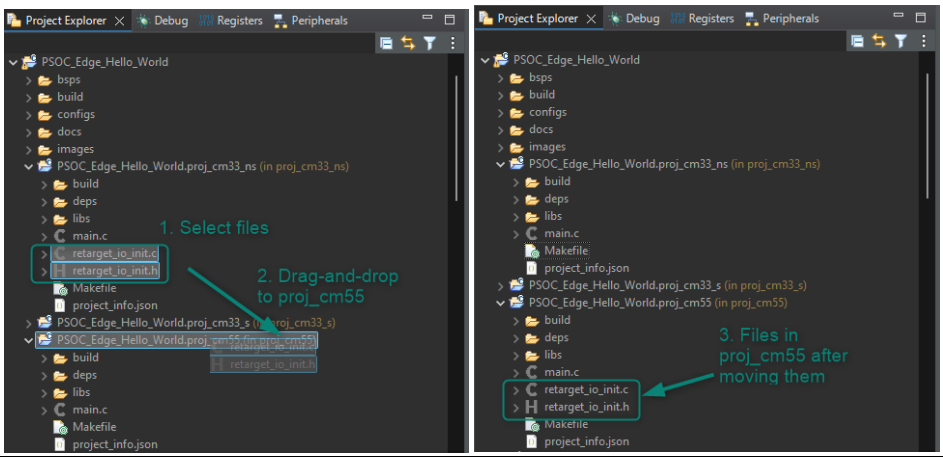



**Figure 19 Move retarget_io_init files from proj_cm33_ns to proj_cm55**

8. Open proj_cm33_ns/main.c and modify it to avoid using retarget-io and printf

**Code listing 2 Remove retarget_io_init.h from cm33_ns**

```
/*******************************************************************************
* Header Files
*******************************************************************************/
#include "cybsp.h"
//#include "retarget_io_init.h"
```

**Code listing 3 Remove retarget-io and printf in main function of cm33_ns**

```
int main(void)
{
    cy_rslt_t result = CY_RSLT_SUCCESS;

    /* Initialize the device and board peripherals. */
    result = cybsp_init();

    /* Board initialization failed. Stop program execution. */
    if (CY_RSLT_SUCCESS != result)
    {
        __disable_irq();    /* Disable all interrupts. */
        CY_ASSERT(0);
        while(true);        /* Infinite loop */
    }

    /* Enable global interrupts */
    __enable_irq();

    /****  retarget-io initialization and printf removed ****/

    /* Enable CM55. */
    /* CM55_APP_BOOT_ADDR must be updated if CM55 memory layout is changed.*/
    Cy_SysEnableCM55(MXCM55, CM55_APP_BOOT_ADDR, CM55_BOOT_WAIT_TIME_USEC);

    for(;;)
    {
        Cy_GPIO_Inv(CYBSP_USER_LED1_PORT, CYBSP_USER_LED1_PIN);
        Cy_SysLib_Delay(BLINKY_LED_DELAY_MSEC);
    }
}
```

9. Now, initialise retarget-io in proj_cm55/main.c, and use printf to display a message

**Code listing 4 Include retarget_io_init.h in cm55**

```
/*******************************************************************************
* Header File
*******************************************************************************/
#include "cybsp.h"
#include "retarget_io_init.h"
```

**Code listing 5 Initialize retarget-io and use printf in main function of cm55**

```
int main(void)
{
    cy_rslt_t result;

    /* Initialize the device and board peripherals. */
    result = cybsp_init();

    /* Board init failed. Stop program execution. */
    if (CY_RSLT_SUCCESS != result)
    {
        handle_app_error();
    }

    /* Enable global interrupts. */
    __enable_irq();

    init_retarget_io();
     /* \x1b[2J\x1b[;H - ANSI ESC sequence for clear screen */
    printf("\x1b[2J\x1b[;H");
    printf("**********  PSOC Edge MCU: IPC Semaphore Example  ********** \r\n");
    printf("CM55 initialization \r\n\n");
    /* Wait for UART traffic to stop */
    while(!(Cy_SCB_UART_IsTxComplete(CYBSP_DEBUG_UART_HW))) {}
    /* Put the CPU to Deep Sleep. */
    for (;;)
    {
        Cy_SysPm_CpuEnterDeepSleep(CY_SYSPM_WAIT_FOR_INTERRUPT);
    }
}
```

10. Build and program the application

### 5.6.1 Output

After the program runs successfully, open the serial terminal and configure the baud rate to 115200-8-N-1. Then press the kit reset button.

Observe that the CM55 CPU prints the welcome message, while the CM33 CPU is still blinking the LED at approximately 1 Hz.

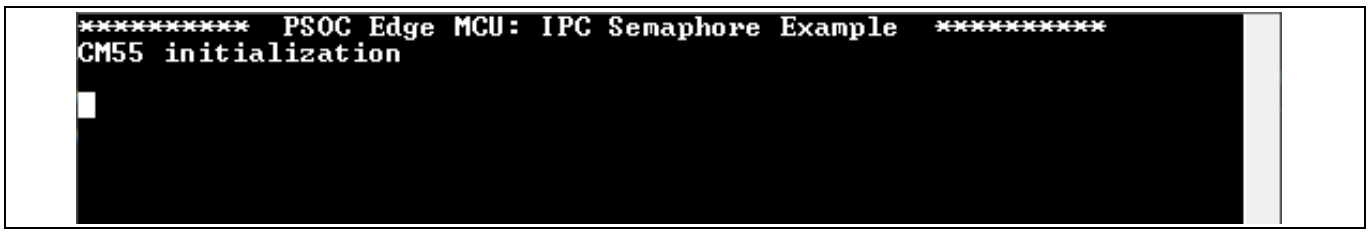



**Figure 20 Output after using retarget-io on CM55**

## 5.7 Implement the GPIO button and print messages using both CPUs

In the previous section, we moved the retarget-io library to CM55; however, it is still possible for the CM33 to utilise the same UART. In this section, we will send UART messages from both CPUs when an EVK button is pressed.

1. Open the **Device Configurator** from the Quick Panel area under the **BSP Configurators** section
2. In the Device Configurator, go to the **Pins** tab
3. In the PSOC™ Edge EVK user buttons are connected to **P8.3** and **P8.7**, per the kit schematic in Figure 21

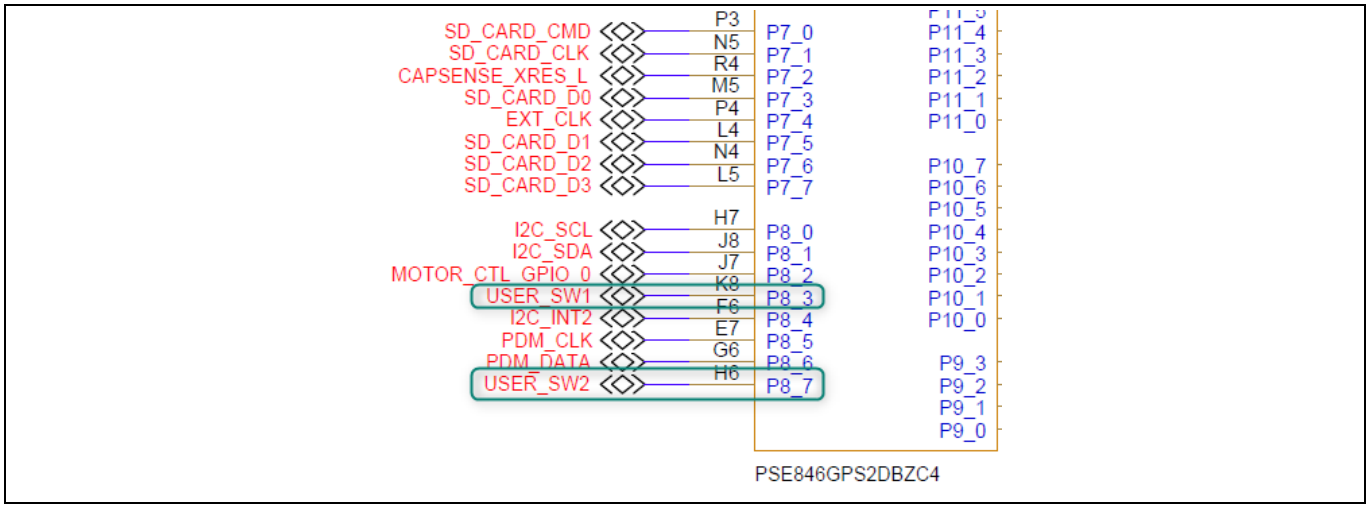



**Figure 21 Connection of buttons in EVK schematics**

4. We will be utilizing USER BTN1 to print messages in our application, which is connected to P8.3.
   Rename **Port8[3]** as **TRAINING_BUTTON**
   Select **File→Save** or press **Ctrl+S** to save design.modus and regenerate source files.

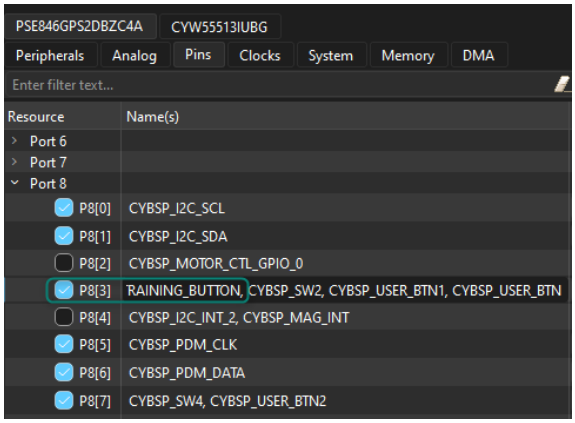



**Figure 22 Renaming P8[3] as TRAINING_BUTTON**

5. Open proj_cm33_ns/main.c and modify the code to use the new button alias to print a message to the terminal

*Note: Observe that we are using a UART PDL function since the CM33 project doesn't include retarget-io. PDL functions for GPIO and UART, which are available in the mtb-dsl-pse8xxgp library.*

**Code listing 6 Update proj_cm33_ns/main.c to print UART message when button is pressed**

```
for(;;)
{
    if (Cy_GPIO_Read(TRAINING_BUTTON_PORT, TRAINING_BUTTON_PIN) == 0)
    {
        Cy_SCB_UART_PutString(CYBSP_DEBUG_UART_HW, "Message sent from CM33 \r\n");
    }
}
```

6. Now, open proj_cm55/main.c and modify the code to implement a similar functionality

*Note: Observe that we are using printf for CM55 since retarget-io was included.*

**Code listing 7 Update proj_cm55 /main.c to print UART message when button is pressed**

```
for (;;)
{
    if (Cy_GPIO_Read(TRAINING_BUTTON_PORT, TRAINING_BUTTON_PIN) == 0)
    {
        printf("Message sent from CM55 \r\n");
    }
}
```

7. Save the files, rebuild the project and program the device

### 5.7.1 Output

After the program runs successfully, open the serial terminal and configure the baud rate to 115200-8-N-1. Then press the kit reset button.

Press USER_BTN1 on the EVK and observe that both devices will attempt to send messages to the terminal; however, the output is incorrect since no semaphores are implemented to synchronize access to UART.


**Figure 23 Output sending messages from both CPUs without semaphores**

## 5.8 Use IPC semaphore to synchronise access to UART

We previously observed that both CPUs can access the UART concurrently; however, this can result in problems when both cores attempt to write at the same time without synchronisation.

In this section, we will use IPC semaphores to synchronise access.

1. Create a shared folder to facilitate sharing parameters between projects.
   In Eclipse's Project Explorer, right-click on the top level of the project and select **New→Folder**, then name the folder as **shared**

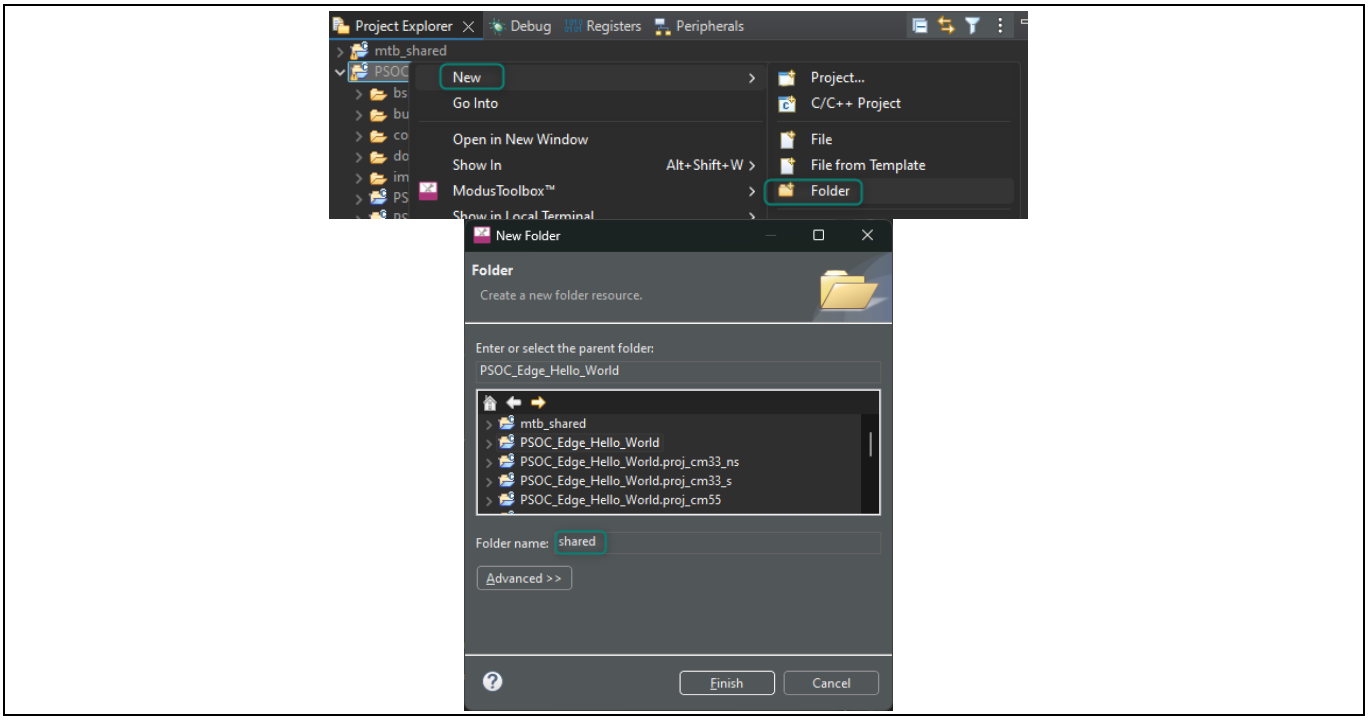



**Figure 24 Create a shared folder**

2. Repeat the process to create an **include** folder inside **shared** folders should look as in Figure 25

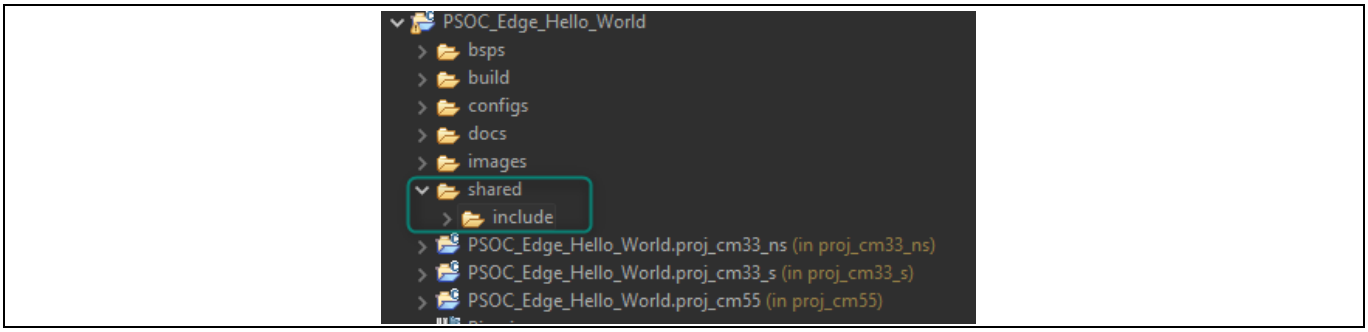



**Figure 25 Shared folders**

3. Right-click in the **shares/include** folder, and select **New→File**, then name the file as **ipc_def.h**

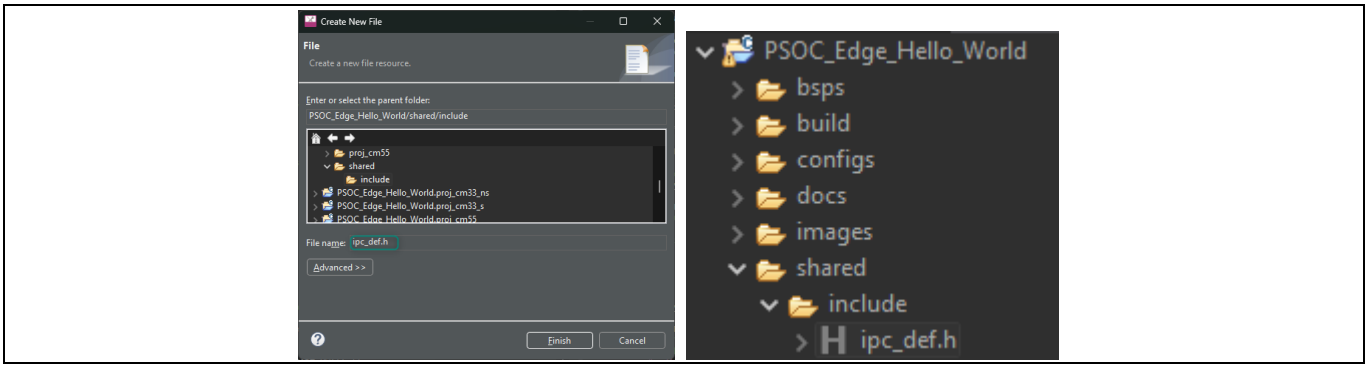



**Figure 26 Create ipc_def.h**

4. Add the following code to the newly created ipc_def.h file

**Code listing 8 ipc_def.h code**

```
#ifndef IPC_DEF_H
#define IPC_DEF_H

    /* IPC Channel to use */
    #define IPC_CHANNEL_NUM         8U

    /* Semaphore number to be used in this example. Semaphores 0-15 are reserved
       for system use. */
    #define MAX_SEMA_NUM            32U
    #define MY_SEMA_NUM             16U

    /* Delay to allow other core to acquire the semaphore */
    #define SEMA_DELAY              10U

#endif /* IPC_DEF_H */
```

5. Add the **shared/include** path to proj_cm33_ns/Makefile and proj_cm55/Makefile

*Note: The Makefile for both projects needs to be updated since we will access ipc_def.h from both projects.*

**Code listing 9 Add shared include path to Makefile of proj_cm33_ns and proj_cm55**

```
# Like SOURCES, but for include directories. Value should be paths to
# directories (without a leading -I).
INCLUDES+=../shared/include
```

6. Open proj_cm33_ns/main.c and add code to initialize the semaphore

*Note: Observe that a region in shared memory is used to declare the semaphore variable since it needs to be accessed by both CPUs.*

**Code listing 10 Include shared file and declare sema_data in proj_cm33_ns/main.c**

```
/*******************************************************************************
* Header Files
*******************************************************************************/
#include "cybsp.h"
#include "ipc_def.h"

/*******************************************************************************
* Global variables
```

```
*****************************************************************************/
CY_SECTION_SHAREDMEM uint32_t sema_data[CY_IPC_SEMA_COUNT / CY_IPC_SEMA_PER_WORD];
```

**Code listing 11 Initialize semaphore in proj_cm33_ns/main.c**

```
int main(void)
{
    cy_rslt_t result = CY_RSLT_SUCCESS;
    cy_en_ipcsema_status_t ipc_status;

    /* Initialize the device and board peripherals. */
    result = cybsp_init();

    /* Board initialization failed. Stop program execution. */
    if (CY_RSLT_SUCCESS != result)
    {
        __disable_irq();    /* Disable all interrupts. */
        CY_ASSERT(0);
        while(true);        /* Infinite loop */
    }

    /* Enable global interrupts */
    __enable_irq();

    /****  retarget-io initialization and printf removed ****/

    ipc_status = Cy_IPC_Sema_Init(IPC_CHANNEL_NUM, CY_IPC_SEMA_COUNT, sema_data);

    if (CY_IPC_SEMA_SUCCESS != ipc_status)
    {
        __disable_irq();    /* Disable all interrupts. */
        CY_ASSERT(0);
        while(true);        /* Infinite loop */
    }

    /* Enable CM55. */
    /* CM55_APP_BOOT_ADDR must be updated if CM55 memory layout is changed.*/
    Cy_SysEnableCM55(MXCM55, CM55_APP_BOOT_ADDR, CM55_BOOT_WAIT_TIME_USEC);
    Cy_SysLib_Delay(SEMA_DELAY);

    for(;;)
```

7. Then, add the code to synchronize access to the UART

**Code listing 12 Use IPC semaphore in proj_cm33_ns/main.c**

```
for(;;)
{
    if (Cy_GPIO_Read(TRAINING_BUTTON_PORT, TRAINING_BUTTON_PIN) == 0)
    {
        /* Acquire semaphore */
        while (Cy_IPC_Sema_Set(MY_SEMA_NUM, false) != CY_IPC_SEMA_SUCCESS)
            ;
        Cy_SCB_UART_PutString(CYBSP_DEBUG_UART_HW, "Message sent from CM33 \r\n");
        while(!(Cy_SCB_UART_IsTxComplete(CYBSP_DEBUG_UART_HW))) {}
        /* Release semaphore */
        while (Cy_IPC_Sema_Clear(MY_SEMA_NUM, false) != CY_IPC_SEMA_SUCCESS)
            ;
    }
}
```

8. Now, open proj_cm55/main.c and add code to initialize the semaphore

*Note: Observe that the sema_data variable does not need to be declared again.*

**Code listing 13 Include shared file in proj_cm55/main.c**

```
/*******************************************************************************
* Header File
*******************************************************************************/
#include "cybsp.h"
#include "retarget_io_init.h"
#include "ipc_def.h"
```

**Code listing 14 Initalize semaphore in proj_cm55/main.c**

```
int main(void)
{
    cy_rslt_t result;
    cy_en_ipcsema_status_t ipc_status;

    /* Initialize the device and board peripherals. */
    result = cybsp_init();

    /* Board init failed. Stop program execution. */
    if (CY_RSLT_SUCCESS != result)
    {
        handle_app_error();
    }

    /* Enable global interrupts. */
    __enable_irq();

    ipc_status = Cy_IPC_Sema_Init(IPC_CHANNEL_NUM, 0UL, NULL);

    if(ipc_status != CY_IPC_SEMA_SUCCESS)
    {
        handle_app_error();
    }

    init_retarget_io();
```

9. Lastly, use the semaphore to synchronize access to UART

**Code listing 15 Use IPC semaphore in proj_cm55/main.c**

```
for (;;)
{
    if (Cy_GPIO_Read(TRAINING_BUTTON_PORT, TRAINING_BUTTON_PIN) == 0)
    {
         /* Acquire semaphore */
        while (Cy_IPC_Sema_Set(MY_SEMA_NUM, false) != CY_IPC_SEMA_SUCCESS)
            ;
        printf("Message sent from CM55 \r\n");
        while(!(Cy_SCB_UART_IsTxComplete(CYBSP_DEBUG_UART_HW))) {}
        /* Release semaphore */
```

```
        while (Cy_IPC_Sema_Clear(MY_SEMA_NUM, false) != CY_IPC_SEMA_SUCCESS)
            ;
    }
}
```

10. Save the files, rebuild the project, and reprogram the device

### 5.8.1 Output

After the program runs successfully, open the serial terminal and configure the baud rate to 115200-8-N-1. Then press the kit reset button.

Press USER_BTN1 on the EVK and observe that both devices will send messages to the terminal and now that IPC semaphores are used to synchronize access, both cores can print the complete message successfully.

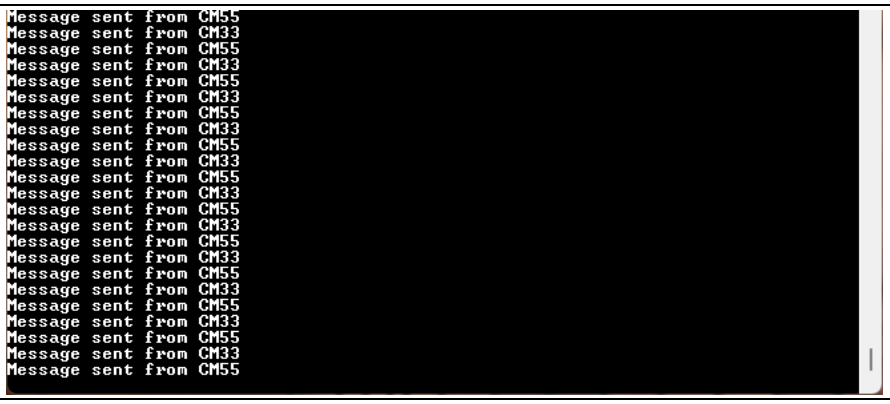



**Figure 27 Output sending messages from both CPUs using IPC semaphores**

## 5.9 Conclusion

We successfully learned how to use the Library Manager to add and remove libraries on both CPUs, implemented code to access a shared resource, and synchronized that access with an IPC semaphore.

This demonstrates IPC's effectiveness for coordinating access between processors, and it can also be used to exchange messages between them.

# 6 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™

The following steps show how to create a new project for PSOC™ Edge in ModusToolbox™.

1. Open the **ModusToolbox™ Dashboard 3.6** and launch the **Eclipse IDE for ModusToolbox™**

   *Note: If you have not installed ModusToolbox™ or Eclipse IDE, see Required development tools section.*

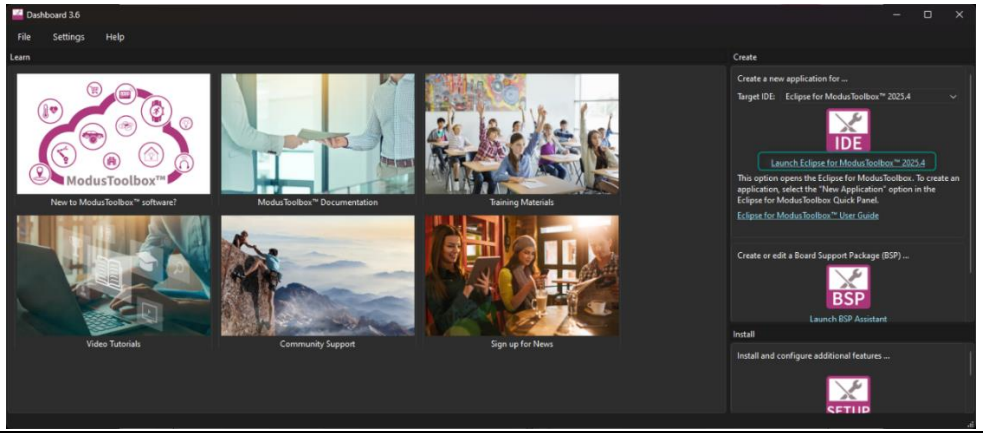



**Figure 28 Open Eclipse IDE for ModusToolbox™ from Dashboard**

2. Choose a **workspace directory** for your project. A folder will be created if the workspace does not exist. After selecting a suitable directory, click the **Launch** button

   *Note: It is recommended to keep the workspace path short. Windows has a 260-character path length limit, and a long workspace path may cause build issues when creating some applications.*

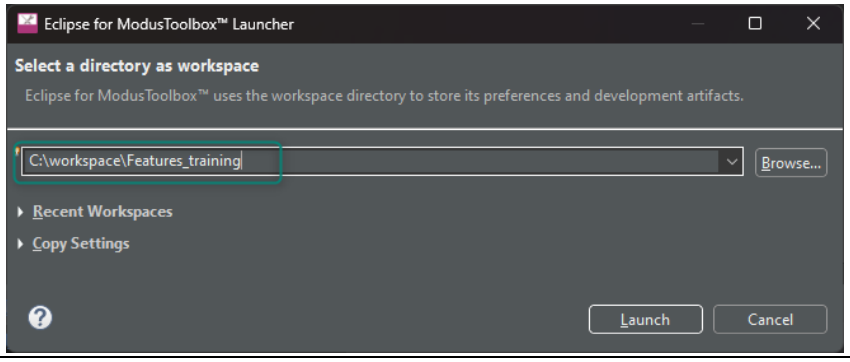



**Figure 29 Choose workspace directory**

3. Select **New Application** from the Quick Panel. Alternatively, go to **File> New> ModusToolbox™** application. This will launch the **Project Creator Tool**

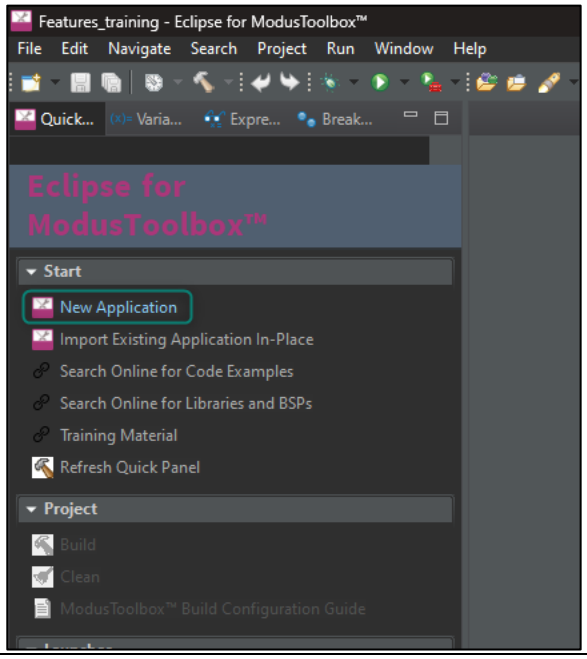



**Figure 30 Create a new application**

4. Once **Projector Creator** is launched, you will be prompted to select a board support package (BSP). Select the **KIT_PSE84_EVAL_EPC2 BSP** under the PSOC™ Edge BSPs and click **Next**

*Note: BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.*
*Observe that ModusToolbox™ also includes BSPs for PSOC™ Edge EPC4 devices (KIT_PSE84_EVAL_EPC4), and for the PSOC™ Edge AI Kit (KIT_PSE84_AI); however, this training is intended to be used with the KIT_PSE84_EVAL, which includes EPC2 devices by default.*

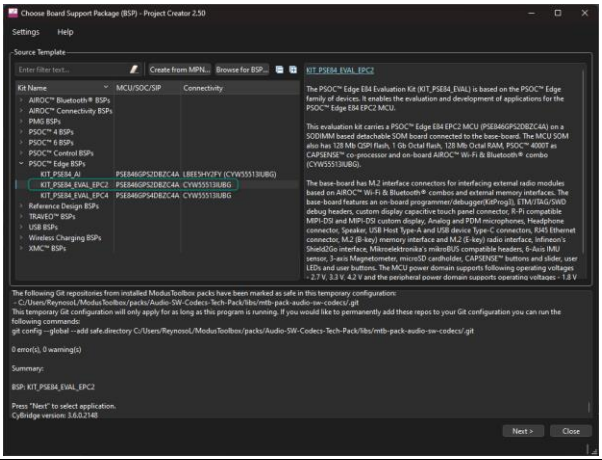



**Figure 31 Selecting BSP in Project Creator**

5. A new window opens to select an application. The Project Creator includes categories and a search field to make it easier to find and select different examples. Click the **checkbox** near the application, optionally rename the project, and then click **Create**. Figure 32 shows the creation of the **PSOC™ Edge Hello World** application under the **Getting Started section**

   *Note: Windows has a 260-character path length limit. A long workspace path and/or a long project name may cause build issues when creating some applications.*

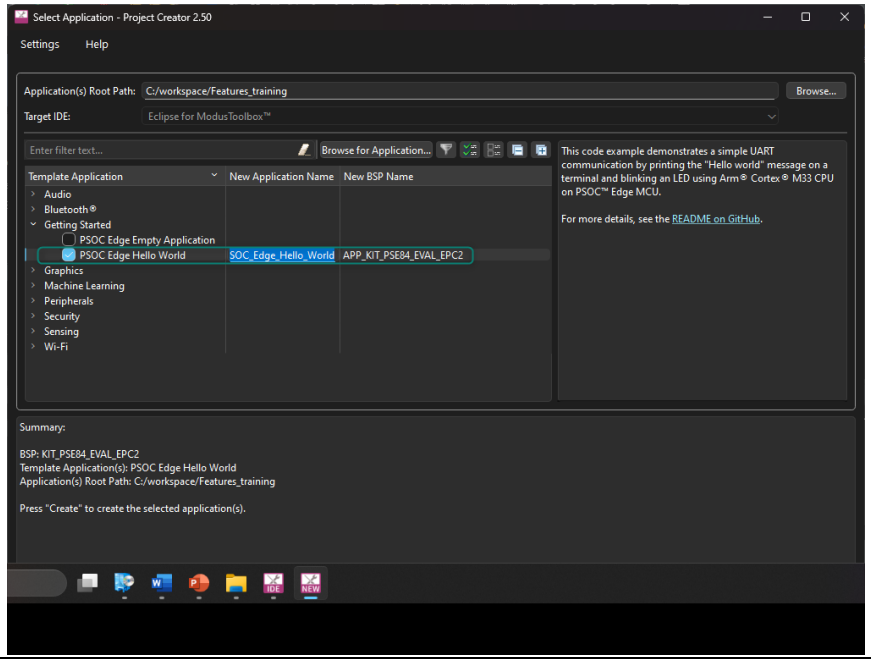



**Figure 32 Selecting application**

6. A new project is being created, as shown in Figure 33, all the files in the Project Explorer window are displayed

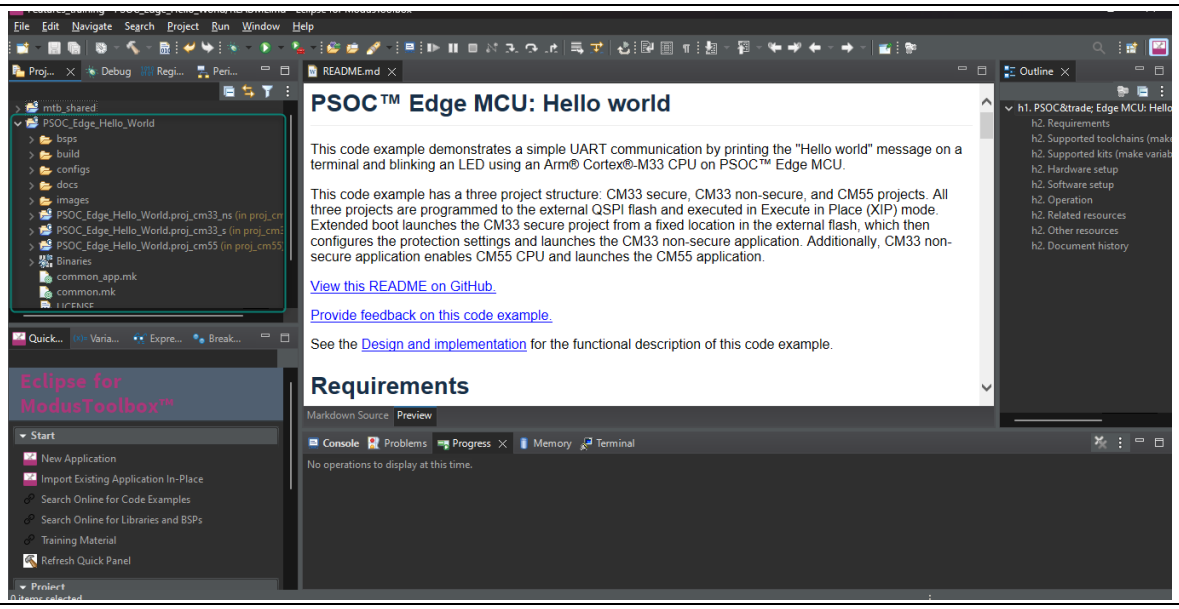



**Figure 33 New application created**

# 7 Appendix B: KIT_PSE84_EVAL details

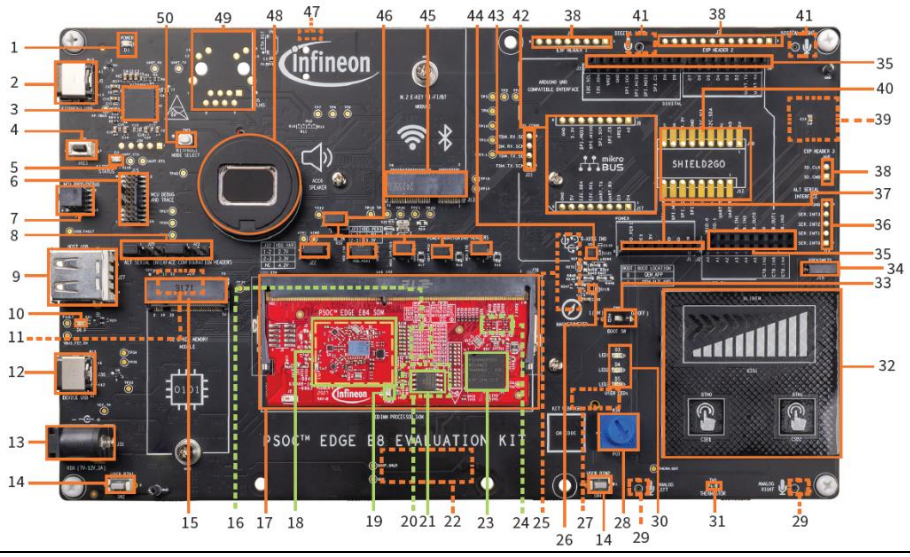



**Figure 34 PSOC™ Edge evaluation kit (EVK) details**

1 Baseboard power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C Power Delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128 Mb Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor system-on-module (SoM) 260-pin SODIMM connector (J28)
18 AIROC™ CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor system-on-module (SoM) power LED (D3)
20 1 Gb Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128 Mb Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi-compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

* Footprint only, not populated on the board
** Component at the bottom side of the Baseboard
*** Component on the bottom side of the SoM

## Revision history

| Document revision | Date | Description of changes |
|---|---|---|
| ** | 2026-03-15 | Initial release. |

**Trademarks**

All referenced product or service names and trademarks are the property of their respective owners.

PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

**Edition 2026-03-15**

**Published by**

**Infineon Technologies AG**

**Am Campeon 1-15**

**85579 Neubiberg**

**Germany**



**Do you have a question about this document?**

**Email:**

erratum@infineon.com

**Important Notice**

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com